DESIGN & ANALYSIS OF ALGORITHMS
PROJECT REPORT SUBMISSION FORM

EVIDENCE SHEET

Assignment (1–6): Assignment 4	**Course / Section:** DAA / AI ML Leopards	**Faculty:** Prof. Geeta Tripathi
Project / Framework Title: Adaptive Network Design and Optimization Framework		
Team Members & Roll Numbers: 1. M. Harshavardhan Reddy (`25WU0102152`) | 2. M. Sai Manoj (`25WU0102139`) | 3. Y. Abhinav Sri Ram (`25WU0101158`) | 4. M. Manoj Sri Sathya Charan (`25WU0102169`) | 5. K. Pranay (`25WU0102328`)		
Code / Repository URL: github.com/abhinavsriram77/Adaptive-Network-Design-and-Optimization-Framework		**Demo Link:** blob/main/main.py

1 PROBLEM ABSTRACTION & MODELLING (Nodes, Edges, Multi-Tier Objectives)

LO1 · model real world scenarios

Scenario / Decision to Solve: Manage a heterogeneous enterprise/city network of **Data Centers**, **Regional Hubs**, and **IoT Gateways**. Dynamically select optimal DAA techniques across 6 operations: minimum-cost backbone construction, all-pairs communication routing, volatile RAM vs. persistent disk state indexing, fault alarm prioritization, capital upgrade budgeting, and field engineer maintenance tours.

Entities / Inputs & Size Parameters: Graph `G = (V, E)` with vertex count `|V|`, link count `|E|`, edge density `D = 2|E| / (|V|(|V|-1))`; telemetry records `N_rec`, B-Tree block degree `t = 16`; fault alarms `N_inc`, escalations `K_esc`; upgrade proposals `n` with cost `w_i`, utility `v_i`, budget `W`; maintenance sites `N_tsp` with latency matrix `C[i][j]`.

Multi-Tier Objectives & Constraints: (1) *MST Backbone:* `Min Sum w(e)` subject to `|T| = |V| - 1` acyclic connectivity. (2) *All-Pairs Routing:* `Min Latency(i -> j)` for all `(i, j) in V×V` with no negative cycles (`D[i][i] >= 0`). (3) *Indexing & Triage:* Ordered `O(log N)` search (`[t-1, 2t-1]` keys/block) and Min-Heap property. (4) *Budgeting:* `Max Sum v_i*x_i` s.t. `Sum w_i*x_i <= W` for continuous `x_i in [0,1]` or discrete `x_i in {0,1}`. (5) *TSP Tour:* `Min Sum C[π(i), π(i+1)] + C[π(N-1), π(0)]` visiting each site once and returning to depot `v_0`.

Representation: `[✓] graph` `[✓] optimization` `[✓] state/search` `[✓] DP` `[✓] data structure`

2 ALGORITHM CHOICE & FRAMEWORK DESIGN (Density D, Divisibility, Memory Tiers)

LO2 · choose & compare

Implemented DAA Syllabus Techniques & Alternatives Compared: (1) **Kruskal's MST** (DSU Path Compression + Rank) vs. **Prim's MST** (Min-Heap & Dense Matrix); (2) **Floyd-Warshall DP**; (3) **Red-Black Tree** vs. **Order-t B-Tree**; (4) **Binomial Heap** vs. **Fibonacci Heap**; (5) **Fractional Knapsack (Greedy)** vs. **0/1 Knapsack (DP)**; (6) **TSP DFS Backtracking** vs. **TSP Branch-and-Bound** (Reduced Cost Matrix LB).

Selection Reason / Runtime Decision Rules:

- **Graph Density Rule (`D`):** Empirical crossover at `D ≈ 0.31` (configured threshold `D = 0.35`). Selects **Kruskal's** (`O(E log E)`) when `D < 0.35`; switches to **Prim's Dense** (`O(V²)`) when `D >= 0.35` to avoid global edge-sorting overhead.
- **Memory Tier & Escalation Rules:** Live RAM telemetry routes to **Red-Black Tree** (`<= 2` rotations/insert); disk logs route to **Order-16 B-Tree** (`O(log_t N)` block reads). Fault queues use **Binomial Heap** when escalation ratio `R_esc < 0.20` and live-migrate to **Fibonacci Heap** (`O(1)` amortized `decrease_key`) when `R_esc >= 0.20`.
- **Resource Divisibility & TSP Scale Rules:** Divisible bandwidth (`x_i in [0,1]`) triggers **Greedy Fractional Knapsack**; indivisible hardware (`x_i in {0,1}`) triggers **0/1 Knapsack DP**. TSP uses **Exact Branch-and-Bound** for `N <= 14` and transitions to bounded polynomial heuristics for `N > 14`.

```
Workflow Pipeline: SCENARIO INPUT → PROBLEM MODEL → DAA CHOICE ENGINE (D, Tier, R_esc, x_i, N) → EXECUTION → TELEMETRY ANALYSIS → LIVE ADAPTATION
```

3 ASYMPTOTIC COMPLEXITY & CORRECTNESS TABLE

LO3 · reason formally

Major Technique / Operation	Expected Time T(n)	Space S(n)	Assumption / Formal Correctness Argument
Kruskal's MST (DSU Rank + Path Comp.)	`O(|E| log |E|)`	`O(|V| + |E|)`	**Cut Property:** Adding lightest safe edge between disjoint sets yields an exact MST.
Prim's MST (Heap / Dense Matrix)	`O(|E| log |V|)` / `O(|V|²)`	`O(|V| + |E|)`	**Cut Property:** Growing single tree via minimum crossing edge is globally optimal.
Floyd-Warshall All-Pairs DP	`Θ(|V|³)`	`Θ(|V|²)`	**Optimal Substructure:** `D^(k)[i][j] = min(D^(k-1)[i][j], D^(k-1)[i][k]+D^(k-1)[k][j])`; no neg. cycles.
Red-Black Tree vs. **Order-t B-Tree**	`O(log₂ N)` vs. `O(log_t N)` I/O	`O(N)`	**Height Invariants:** RBT bounds height `<= 2 log₂(N+1)`; B-Tree splits full `2t-1` nodes on descent.
Binomial vs. **Fibonacci Heap**	`O(log N)` vs. `O(1)` amort.	`O(N)`	**Potential Method:** Cascading cuts bound amortized `decrease_key` & `union` to `O(1)`.
Fractional Knapsack (Greedy)	`O(n log n)` *(Polynomial)*	`O(n)`	**Greedy Exchange Proof:** Sorting by density `v_i / w_i` is optimal for continuous slices `x_i in [0,1]`.
0/1 Knapsack (Dynamic Programming)	`O(n · W)` *(Pseudo-Poly)*	`O(n · W)`	**Bellman Recurrence:** `DP[i][w] = max(DP[i-1][w], v_i + DP[i-1][w-w_i])` over discrete states.
TSP Backtracking vs. **Branch & Bound**	`O((N-1)!)` worst-case	`O(N²)` per state	**Admissible Lower Bound:** Matrix reduction satisfies `LB <= OPT`; pruning `LB >= UB` preserves optimum.

Complexity Status: `[✓] polynomial` `[✓] pseudo-polynomial` `[✓] NP-Hard optimization` `[✓] NP-Complete decision`

4 IMPLEMENTATION VERIFICATION (28/28 Core Milestone & 42/42 Full Test Suite Passed)

LO4 · build & verify

Language / Tools & Implemented Modules: Python 3.13, `pytest`, modular packages: `core/` (Node, Edge, Graph), `algorithms/mst/` (DisjointSet, Kruskal, Prim, Selector), `algorithms/routing/` (FloydWarshall), `indexing/` (RedBlackTree, BTree, Selector), `prioritization/` (BinomialHeap, FibonacciHeap, Selector), `budgeting/` (Fractional, Discrete01DP, Selector), `dispatch/` (Backtracking, BranchAndBound, ReducedMatrix, Selector), `benchmarks/`, and interactive CLI `main.py`.

Correctness Tests: `[✓] normal` `[✓] boundary` `[✓] large` `[✓] difficult` | **Tests Passed: 28 / 28 Core Milestone Tests** (**42 / 42 Total Extended Tests Passed** in `0.38s`) | **Code Proof:** `tests/` & `main.py`

Test Family Coverage: Validated single-vertex & disconnected graphs (`GraphDisconnectedError`), negative-cycle detection (`NegativeCycleError`), Red-Black black-height & B-Tree degree invariants on sequential/random keys, Fibonacci cascading cuts, 0/1 Knapsack greedy-failure counterexamples, and exact TSP B&B equivalence against brute-force permutations.

5 EXPERIMENTS & ADAPTATION BENCHMARKS (Sparse vs. Dense Crossover at D = 0.31)

LO5 · measure & adapt

Input / Scenario Condition	Size n	Selected Algorithm / Structure	Runtime / Ops / Memory	Result / Empirical Observation
1. Baseline (Sparse Topology)	`|V|=25, |E|=36` (`D=0.12`)	**Kruskal's MST (DSU)**	`0.23 ms / 258 ops`	Optimal MST cost `420.40`; minimal edge sorting cost.
1. Changed (Dense Mesh)	`|V|=25, |E|=240` (`D=0.80`)	**Prim's Dense Matrix MST**	`0.31 ms / 1,250 ops`	**Crossover at `D ≈ 0.31`**; avoids sorting `O(V²)` edges.
2. All-Pairs Scaling	`|V| = 10 → 20 → 30`	**Floyd-Warshall DP**	`1,000 → 8,000 → 27,000` ops	Confirms exact cubic `Θ(|V|³)` growth (`8×` per `2×` nodes).
3. Baseline vs. Changed Index	`N = 500` records (`t=16`)	**Red-Black (RAM) → B-Tree (Disk)**	`485 rot.` vs. `30 splits`	B-Tree compresses height to 2 levels for block I/O.
4. Baseline vs. Storm Triage	`N=50, R_esc: 0.04 → 0.34`	**Binomial → Fibonacci Heap**	`50 cmp → O(1) amort. cuts`	Live queue migration triggered at `R_esc >= 0.20`.
5. Divisible vs. Discrete CapEx	`W = $100k` (`[0,1]` vs `{0,1}`)	**Fractional Greedy → 0/1 DP**	`3 ops` vs. `404 DP states`	Greedy achieves `540.0` (BW); DP achieves `370.0` (HW).
6. Baseline (TSP Unpruned)	`N = 8` sites (FW matrix)	**Exact DFS Backtracking**	`7.27 ms` / **`6,171 states`**	Finds exact optimal tour `451.30 ms` via deep recursion.
6. Changed (TSP Pruned B&B)	`N = 8` sites (FW matrix)	**Exact Branch-and-Bound**	`4.52 ms` / **`45 states`** (`139 pruned`)	Identical `451.30 ms` tour with **`99.2%` fewer states**!

Decision / Behaviour Changed Because: Crossing structural thresholds—Edge Density `D >= 0.31` (configured at `0.35`), Disk Block persistence, Alarm Escalation Ratio `R_esc >= 0.20`, Discrete `x_i in {0,1}` constraint, and TSP cluster size `N > 14`—alters the dominant asymptotic term. Evidence: `benchmarks/benchmark_*.py` & `main.py`.

6 EXPLANATION, LIMITATIONS & NP-HARD / NP-COMPLETE SCALABILITY DISCUSSION

LO5 · interpret & communicate

Main Finding & Scalability Limitation: Adaptive selection consistently outperforms any static algorithm across heterogeneous workloads. Notably, Reduced-Matrix Branch-and-Bound prunes `85%–99%` of the TSP state space compared to Backtracking. **Key Limitations:** (1) Floyd-Warshall incurs rigid `Θ(|V|³)` cost even on sparse graphs; (2) 0/1 Knapsack DP table memory `O(n · W)` grows linearly with numeric budget magnitude `W`; (3) Fibonacci Heaps carry 4 pointers/node, creating higher constant cache overhead on tiny queues.

Theory vs. Observation & Gaps Enabling Modern Algorithms: Empirical operation counts align with theoretical growth curves, while classic DAA gaps directly motivated modern algorithms: Floyd-Warshall's sparse inefficiency led to **Johnson's Algorithm (`O(V² log V + VE)`)** and **Contraction Hierarchies**; B-Tree random write amplification on SSDs led to **LSM-Trees**; Fibonacci Heap pointer overhead led to **Pairing Heaps**; and 0/1 Knapsack capacity blowup led to **FPTAS value scaling**.

Precise NP-Hard / NP-Complete & Pseudo-Polynomial Distinctions:

- **Optimization TSP vs. Decision TSP:** *Optimization TSP* ("Find the minimum-cost Hamiltonian tour") is **NP-Hard** (not a decision problem). *Decision TSP* ("Does a tour of total cost `<= K` exist?") is **NP-Complete** (verifiable in `O(V)` polynomial time and NP-Hard via reduction from Hamiltonian Cycle). Because exact search scales as `O((N-1)!)` (for `N = 22`, `21! ≈ 5.1 × 10¹⁹` permutations), exact methods are restricted to `N <= 14–18` before handing off to bounded heuristics.
- **0/1 Knapsack Pseudo-Polynomiality:** 0/1 Knapsack DP runs in `O(n · W)` time. Since numeric capacity `W` is encoded in `b = log₂(W)` bits, runtime with respect to input bit-length is `O(n · 2^b)` (exponential in `b`, hence **Pseudo-Polynomial**), whereas its decision form is **NP-Complete** and Fractional Knapsack (`O(n log n)`) is strictly **Polynomial**.

SUBMISSION CHECK: `[✓] integrated framework` `[✓] working code` `[✓] alternative compared` `[✓] time/space analysis` `[✓] 4 test families` `[✓] adapted decision` `[✓] report + demo`

ASSESSOR: Modelling __/5 | Selection __/5 | Analysis __/5 | Code __/5 | Experiments __/5 | Demo __/5 **TOTAL __/30**

Faculty Feedback: _____